Actividad Seguridad

🧩 1. Conceptos base (definir con precisión)

Completar tabla:

Concepto	Definición Simple	Nivel Técnico Breve	Ejemplo Real
Permissions	Quién puede entrar a qué "habitación" y qué puede hacer ahí (leer o borrar).	Control de acceso (RBAC/ACL) que define privilegios sobre recursos del sistema.	Un usuario puede ver su perfil (Read), pero solo un Admin puede borrarlo (Delete).
Password Segura	Una llave compleja que es casi imposible de adivinar o forzar.	Cadena de alta entropía con mezcla de caracteres, almacenada con *hashing* y *salt*.	Kj8#pL2$mN1!Q es segura; password123 se rompe en milisegundos por fuerza bruta.
Vulnerabilidad	Una "ventana abierta" o un fallo en el código que un atacante puede aprovechar.	Punto débil en el diseño o implementación de un software (ej. un bug no parcheado).	Un formulario que no limpia el texto y permite inyectar código malicioso (SQL Injection).
Intrusión	Cuando alguien entra a un sistema o red sin tener permiso para estar ahí.	Acceso no autorizado que viola la política de seguridad mediante el uso de un *exploit*.	Un hacker que logra entrar al panel de administración usando una sesión robada.
Seguridad de Red	El conjunto de muros y guardias que protegen el flujo	Uso de Firewalls, VPNs y segmentación para proteger la	Un **Firewall** que bloquea todas las conexiones entrantes

	de datos entre computadoras.	integridad y privacidad del tráfico.	excepto las del puerto 443 (HTTPS).

🔑 2. Permissions (accesos en sistemas)

Responder:

- ¿Qué significa:
  - Autenticación?
  - Autorización?
- ¿Cuál es la diferencia entre ambas?
- Ejemplo developer:
  - Login → ¿qué es?
  - Roles (admin/user) → ¿qué es?

¿Qué significa cada una?

- **Autenticación (AuthN):** Es el proceso de verificar **quién eres**. Es el acto de demostrar tu identidad ante el sistema.
- **Autorización (AuthZ):** Es el proceso de verificar **qué puedes hacer**. Una vez que el sistema sabe quién eres, revisa si tienes permiso para entrar a una carpeta o borrar un dato.

¿Cuál es la diferencia entre ambas?

La **Autenticación** es la llave que te permite entrar al edificio; la **Autorización** es el permiso que dice si puedes entrar a la oficina del jefe o solo al comedor.

👉 Ejemplo Developer: El flujo en tu App

1. **Login (Autenticación):** * **Qué es:** Cuando el usuario ingresa su email y contraseña, y tu backend genera un **Token (JWT)** o una **Sesión**.
   - **Resultado:** El sistema dice: "Confirmado, eres el usuario 'Juan'".
2. **Roles (Admin / User) (Autorización):** * **Qué es:** Es la lógica de negocio que dice: `if (user.role == 'admin') { allow_delete() }`.
   - **Resultado:** El sistema dice: "Juan es un 'User' estándar, así que puede leer el post pero no puede borrarlo".

🔒 3. A Good Password (lo mínimo obligatorio)

Responder:

- ¿Qué hace que una contraseña sea segura?
- ¿Por qué NO se deben guardar contraseñas en texto plano?
- ¿Qué es hashing?

👉 Ejemplo developer:

- ¿Cómo se manejan contraseñas en backend?

🛡️ Gestión de Credenciales: Seguridad Nivel Pro

¿Qué hace que una contraseña sea segura?

No es solo la longitud, es la **entropía** (la aleatoriedad). Una contraseña segura combina:

- **Longitud:** Mínimo 12-16 caracteres.
- **Complejidad:** Mezcla de mayúsculas, minúsculas, números y símbolos.
- **Impredictibilidad:** No usar palabras de diccionario, fechas de nacimiento o nombres de mascotas.

¿Por qué NO se deben guardar en texto plano?

Si guardas `password123` tal cual en tu base de datos y un atacante obtiene acceso a ella (por una intrusión), **todas las cuentas quedan expuestas inmediatamente.** Además, muchos usuarios repiten contraseñas en otros sitios (Gmail, bancos), por lo que el daño sería masivo.

¿Qué es Hashing?

Es una **función criptográfica de una sola vía** que convierte cualquier texto en una cadena de caracteres fija e irreconocible.

- **Propiedad clave:** No se puede "des-hashear". No existe un botón de "reversa" para volver del hash a la contraseña original.
- **Salt (Sal):** Se añade un texto aleatorio extra antes de hashear para que dos usuarios con la misma contraseña tengan hashes diferentes.

👉 Ejemplo Developer: Manejo en el Backend

Cuando un usuario se registra o hace login, el flujo correcto es:

1. **Registro:**
    - El usuario envía `mi_clave_segura`.

    - El backend usa una librería (como **bcrypt** o **argon2**).
    - Se genera un hash: `$2b$12$Kj8#pL2...`
    - **Solo el hash** se guarda en la base de datos.
2. **Login:**
    - El usuario envía `mi_clave_segura` de nuevo.
    - El backend toma esa entrada, le aplica el mismo algoritmo de hash y **compara los resultados**.
    - Si los hashes coinciden, el login es exitoso. **El servidor nunca supo cuál era la clave.**

⚠️ 4. Vulnerabilities (fallas reales)

Responder:

- **¿Qué es una vulnerabilidad?**
- **Mencionar al menos 3 tipos comunes:**
    - **SQL Injection**
    - **XSS**
    - **Exposición de datos**

👉 **Ejemplo real:**

- **¿Cómo una mala validación rompe una app?**

⚠️ 4. Vulnerabilidades: Las grietas del sistema

¿Qué es una vulnerabilidad?

Es una debilidad o fallo en el diseño, implementación o administración de un software que puede ser aprovechada por un atacante para comprometer la **confidencialidad**, **integridad** o **disponibilidad** del sistema.

3 Tipos Comunes de Vulnerabilidades

1. **SQL Injection (Inyección SQL):**
    - **Qué es:** El atacante inserta código SQL malicioso en un formulario (ej. el campo de búsqueda).
    - **Riesgo:** Puede saltarse el login, leer toda la base de datos o borrar tablas completas.
2. **XSS (Cross-Site Scripting):**
    - **Qué es:** El atacante inyecta scripts (JavaScript) en páginas web que otros usuarios ven.
    - **Riesgo:** Robar cookies de sesión, redirigir a sitios falsos o capturar lo que el usuario escribe.
3. **Exposición de Datos Sensibles:**

- **Qué es:** Fallo al proteger información crítica como tarjetas de crédito, contraseñas o datos de salud.
- **Riesgo:** Robo de identidad y multas legales graves por falta de cifrado en reposo o en tránsito.

🚨 5. Intrusión (ataques)

Responder:

- ¿Qué es una intrusión?
- ¿Qué busca un atacante?
- ¿Qué pasa si una app es comprometida?

👉 Ejemplo:

- Robo de datos
- Acceso no autorizado

🚨 5. Intrusión: El acceso no autorizado

¿Qué es una intrusión?

Es el acto de **ingresar o permanecer** en un sistema informático, red o aplicación sin tener el permiso legal o técnico para hacerlo. Ocurre cuando un atacante logra evadir las medidas de seguridad (como firewalls o autenticación) utilizando un fallo en el sistema.

¿Qué busca un atacante? (Los 3 pilares)

Generalmente, los objetivos se dividen en:

- **Confidencialidad:** Robar datos (clientes, secretos industriales, tarjetas).
- **Integridad:** Alterar información (cambiar saldos bancarios, modificar registros).
- **Disponibilidad:** Bloquear el servicio para pedir un rescate (**Ransomware**) o simplemente causar daño.

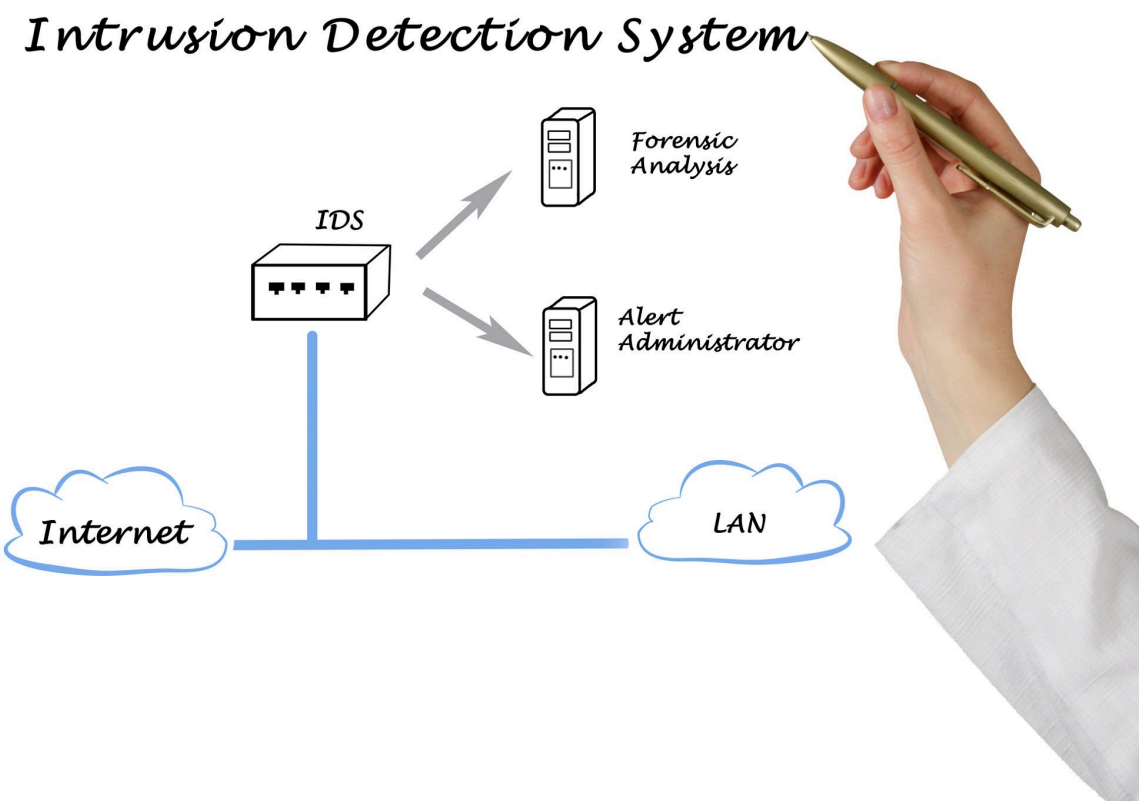
¿Qué pasa si una app es comprometida?

Las consecuencias suelen ser una reacción en cadena:

1. **Filtración de Datos:** Información privada termina en foros de hackers.
2. **Pérdida de Confianza:** Los usuarios dejan de usar la app por miedo.
3. **Consecuencias Legales:** Multas millonarias por no proteger los datos de los usuarios.
4. **Escalada de Privilegios:** El atacante usa la app comprometida como puente para entrar a servidores más importantes de la empresa.

Ejemplo: Robo de datos vs. Acceso no autorizado

- **Acceso No Autorizado:** Un ex-empleado usa sus viejas credenciales (que no fueron borradas) para entrar al panel de administración. Aún no ha robado nada, pero ya violó el sistema.
- **Robo de Datos (Exfiltración):** Un atacante usa un script para descargar automáticamente los 10 millones de filas de tu tabla de `users`. Los datos viajan por la red hacia el servidor del atacante.



🌐 6. Network Security (seguridad en red)

Responder:

- ¿Qué protege la seguridad de red?
- ¿Qué es un firewall?
- ¿Qué es HTTPS y por qué es importante?

👉 Ejemplo developer:

- ¿Por qué usar HTTPS en APIs?

🌐 6. Network Security: Blindando el camino

¿Qué protege la seguridad de red?

Protege la **infraestructura** y los **datos en tránsito**. Su objetivo es evitar que la información sea interceptada, modificada o bloqueada mientras viaja desde el dispositivo del usuario hasta tu servidor. Se enfoca en la "tubería" por donde pasan los bits.

¿Qué es un Firewall?

Es una **barrera de seguridad** (física o lógica) que inspecciona el tráfico de red entrante y saliente. Decide qué paquetes pasan y cuáles se bloquean basándose en reglas predefinidas (como "solo permitir tráfico en el puerto 443" o "bloquear la IP X.X.X.X").

- **Analogía:** Es el guardia de seguridad en la puerta de un edificio que revisa identificaciones antes de dejar entrar a alguien.

¿Qué es HTTPS y por qué es importante?

HTTPS (**HyperText Transfer Protocol Secure**) es la versión segura de HTTP. Utiliza el protocolo **TLS** para cifrar la comunicación.

- **Importancia:** Evita ataques de *Man-in-the-Middle* (donde alguien "escucha" la conexión). Sin HTTPS, cualquier persona en la misma red Wi-Fi podría ver tus contraseñas o datos bancarios en texto plano.

👉 Ejemplo Developer: ¿Por qué usar HTTPS en APIs?

Incluso si tu API no maneja tarjetas de crédito, **HTTPS es obligatorio** por tres razones:

1. **Integridad:** Garantiza que los datos que el backend envía al frontend no han sido modificados por un tercero en el camino.
2. **Tokens de Acceso:** Si usas autenticación (como Bearer Tokens o JWT), estos viajan en los *headers*. Sin HTTPS, cualquier hacker puede robar el token y suplantar al usuario.
3. **SEO y Navegadores:** Los navegadores modernos marcan los sitios HTTP como "No seguros" y los motores de búsqueda penalizan el ranking de las APIs que no usan cifrado.

💻 7. Conexión directa con desarrollo (CLAVE)

Responder:

- ¿Qué errores comunes cometen los developers en seguridad?
- ¿Qué pasaría si:
  - No validas inputs?
  - No proteges rutas?
  - No usas HTTPS?

💻 7. Seguridad en Desarrollo: El Código Blindado

¿Qué errores comunes cometen los developers en seguridad?

- **Hardcoding de Secretos:** Guardar contraseñas de bases de datos o llaves de API directamente en el código (en lugar de usar archivos `.env`).
- **Confianza Ciega en el Frontend:** Pensar que porque el formulario tiene validaciones en JavaScript, el usuario no puede enviar datos maliciosos directamente al backend (usando herramientas como Postman).
- **No actualizar dependencias:** Usar librerías de `npm` o `pip` antiguas que tienen vulnerabilidades conocidas y ya parchadas.
- **Mensajes de Error demasiado detallados:** Mostrar el error completo de la base de datos al usuario, dándole pistas al atacante sobre la estructura de tu tabla.

¿Qué pasaría si...? (Los 3 Pecados Capitales)

1. No validas inputs?

- **Consecuencia:** Tu app es vulnerable a **SQL Injection** y **XSS**.
- **El desastre:** Un usuario podría escribir `; DROP TABLE users;` en un campo de texto y borrar toda tu base de datos, o inyectar un script que robe las cookies de todos tus clientes.

2. No proteges rutas? (Broken Access Control)

- **Consecuencia:** Cualquier persona que adivine una URL (ej: `tu-app.com/admin/delete-user/10`) podrá ejecutar acciones sensibles.
- **El desastre:** Un usuario estándar podría acceder a funciones de administrador simplemente cambiando el ID en la barra de direcciones, exponiendo o borrando información privada.

3. No usas HTTPS?

- **Consecuencia:** Los datos viajan en "texto plano" (como una postal que cualquiera puede leer).
- **El desastre:** Si alguien está en la misma red (ej: el Wi-Fi de un café), puede capturar el **Token de Login** del usuario y suplantar su identidad sin siquiera saber su contraseña.

🌍 8. Caso práctico real

Escenario:

> “Un usuario logra acceder a información que no le corresponde.”

Responder:

- ¿Qué falló?
- ¿Es problema de permisos, vulnerabilidad o intrusión?
- ¿Cómo lo evitarías como developer?

🌍 8. Caso Práctico: El Acceso Indebido

1. ¿Qué falló?

Lo que falló fue el **Control de Acceso (Broken Access Control)**. El sistema autenticó al usuario (sabe quién es), pero no validó correctamente su **autorización** (qué tiene permitido hacer).

- **El error clásico:** El backend confió en el ID que el usuario envió desde el navegador (ej: `GET /api/factura/500`) sin verificar si la factura #500 realmente le pertenece a ese usuario.

2. ¿Es problema de permisos, vulnerabilidad o intrusión?

En este escenario, se cruzan los tres conceptos:

- **Vulnerabilidad:** Existe un fallo en el código (IDOR - Insecure Direct Object Reference) que permite manipular parámetros.
- **Permisos:** La lógica de **Autorización** está mal configurada o es inexistente para esa ruta específica.
- **Intrusión:** El usuario, al actuar con malicia para ver datos ajenos, está realizando una **intrusión lógica** (aunque ya tenga una cuenta válida en la app).

---

3. ¿Cómo lo evitarías como developer? (Checklist de Oro)

1. **Nunca confíes en el ID del Frontend:** No asumas que porque un usuario está logueado puede ver cualquier recurso. Siempre verifica la propiedad en la base de datos:
   - ❌ `SELECT * FROM facturas WHERE id = 500`
   - ✅ `SELECT * FROM facturas WHERE id = 500 AND usuario_id = <ID_DEL_TOKEN>`
2. **Middleware de Autorización:** Implementa una capa de seguridad en tus rutas que verifique los **Roles** (Admin, Editor, User) antes de ejecutar la lógica de negocio.

3. **UUIDs en lugar de IDs secuenciales:** Usa identificadores largos y aleatorios (ej: `f47ac10b-58cc...`) en lugar de números simples (1, 2, 3). Esto evita que un atacante simplemente "sume 1" a la URL para ver el siguiente registro.
4. **Principio de Menor Privilegio:** Por defecto, nadie tiene acceso a nada. Solo abres los permisos necesarios para que cada rol cumpla su función.

🧪 9. Analogía obligatoria

Crear analogía:

Para que estos conceptos de seguridad se queden grabados en tu "disco duro" mental, vamos a usar la **Analogía de la Bóveda Bancaria**.

Aquí tienes el cierre perfecto para tu sección de seguridad:

🏦 El Banco de Datos: Analogía de Seguridad

🔑 Password Segura (La Llave Maestra)

Imagina que la puerta de la bóveda tiene una combinación. Una **password segura** no es "1234", sino una secuencia compleja de giros, ritmos y códigos que solo tú conoces. El **Hashing** es como si el banco no guardara tu combinación, sino una foto del mecanismo interno después de que metes la llave; así, aunque alguien robe la foto, no puede saber cómo es la llave original.

🛡️ Permissions (El Carnet y las Llaves de Paso)

- **Autenticación:** Es el guardia que te pide el DNI en la entrada para confirmar que eres el cliente "Juan".
- **Autorización:** Es el sistema que dice: "Juan puede abrir su cajita de seguridad (User), pero no puede entrar a la oficina del gerente ni abrir la bóveda principal (Admin)".

🪟 Vulnerabilidad (La Ventana Mal Cerrada)

Es ese fallo en el diseño del edificio: quizás una rejilla de ventilación es lo suficientemente grande para que alguien pase, o el sensor de movimiento tiene un "punto ciego". En código, es ese **input que no validaste** y que deja una rendija abierta para un atacante.

👤 Intrusión (El Robo en Progreso)

Es el acto físico de que alguien que no debería estar ahí cruce la línea. El intruso detectó la **vulnerabilidad** (la ventana) y la usó para entrar al pasillo. Ya no es solo un riesgo, es un evento real donde la privacidad del banco ha sido violada.

🌐 Seguridad de Red (El Perímetro y los Sensores)

Es todo lo que rodea al banco: las cámaras de seguridad, los muros de concreto (**VPC/Subnets**) y el guardia que revisa los maletines en la puerta (**Firewall**). El **HTTPS** es como si el dinero viajara en un camión blindado con vidrios polarizados: nadie desde afuera puede ver cuánto hay ni intentar robárselo en el semáforo.

bonus

🎟️ JWT (JSON Web Token)

Es un estándar abierto que permite transmitir información entre dos partes de forma segura y compacta como un objeto JSON.

- **Estructura (Las 3 partes): Se ve como una cadena larga de texto dividida por puntos (header.payload.signature).**
    1. **Header: Indica el algoritmo de cifrado.**
    2. **Payload: Contiene los datos del usuario (id, nombre, rol). ¡Ojo! No es secreto, solo está codificado; no pongas contraseñas aquí.**
    3. **Signature: Es el sello de garantía. El servidor lo firma con una clave secreta para asegurar que nadie alteró el token en el camino.**

🔑 Autenticación con Tokens

Es un método "sin estado" (stateless) donde el servidor no necesita guardar una sesión en su memoria.

1. **El intercambio: El usuario envía sus credenciales, el servidor las valida y le entrega un Token (como un pase VIP de un concierto).**
2. **El uso: En cada petición siguiente, el navegador envía ese token en el *Header* de la petición (Authorization: Bearer <token>).**
3. **La ventaja: Es ideal para apps móviles y sistemas distribuidos, ya que cualquier servidor de tu red puede validar el token sin consultar una base de datos central de sesiones.**

🚦 Rate Limiting (Limitación de Tasa)

Es una técnica de seguridad que controla la cantidad de peticiones que un usuario o una IP puede hacer a tu API en un tiempo determinado.

- **¿Para qué sirve?**
  - **Evitar ataques de Fuerza Bruta: Que nadie intente 10,000 contraseñas por segundo.**
  - **Prevenir ataques DoS: Que un solo usuario no sature tu servidor y lo deje fuera de servicio.**
  - **Costos: Si usas servicios pagados (como la API de OpenAI o Google Maps), evita que un error en tu código o un atacante te dejen una factura millonaria.**
- **Ejemplo: "Solo permitimos 100 peticiones por minuto por cada dirección IP".**

📋 Resumen para el Developer

Concepto	¿Qué es en 1 frase?	Analogía
JWT	El formato del pase de seguridad.	Una credencial con tu foto y sello oficial.
Token Auth	El sistema de usar pases en lugar de llaves.	Usar una pulsera en un hotel All-Inclusive.
Rate Limiting	El guardia que cuenta cuántos entran.	El torniquete del metro que solo deja pasar a uno a la vez.